Supervised Learning and K Nearest Neighbors Exercises

Introduction

We will be using customer churn data from the telecom industry for the first week's exercises. The data file is called `Orange_Telecom_Churn_Data.csv`. We will load this data together, do some preprocessing, and use K-nearest neighbors to predict customer churn based on account characteristics.

```
from __future__ import print_function
import os
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Question 1

- Begin by importing the data. Examine the columns and data.
- Notice that the data contains a state, area code, and phone number. Do you think these are good features to use when building a machine learning model? Why or why not?

We will not be using them, so they can be dropped from the data.

```
df = pd.read_csv("orange_telecom_churn_data.csv")
df.head(5)
```

```
{"type":"dataframe","variable_name":"df"}
```

```
df.shape
```

```
(5000, 21)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 21 columns):
 #   Column                          Non-Null Count  Dtype
---  ------                          --------------  -----
 0   state                           5000 non-null   object
 1   account_length                  5000 non-null   int64
 2   area_code                       5000 non-null   int64
 3   phone_number                    5000 non-null   object
 4   intl_plan                       5000 non-null   object
 5   voice_mail_plan                 5000 non-null   object
```

```
 6   number_vmail_messages          5000 non-null   int64
 7   total_day_minutes              5000 non-null   float64
 8   total_day_calls                5000 non-null   int64
 9   total_day_charge               5000 non-null   float64
 10  total_eve_minutes              5000 non-null   float64
 11  total_eve_calls                5000 non-null   int64
 12  total_eve_charge               5000 non-null   float64
 13  total_night_minutes            5000 non-null   float64
 14  total_night_calls              5000 non-null   int64
 15  total_night_charge             5000 non-null   float64
 16  total_intl_minutes             5000 non-null   float64
 17  total_intl_calls               5000 non-null   int64
 18  total_intl_charge              5000 non-null   float64
 19  number_customer_service_calls  5000 non-null   int64
 20  churned                        5000 non-null   bool
dtypes: bool(1), float64(8), int64(8), object(4)
memory usage: 786.3+ KB
```

```
df.iloc[[0]].T
```


```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 21,\n  \"fields\": [\n{\n      \"column\": 0,\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 21,\n        \"samples\": [\n          \"KS\",\n          3,\n          11.01\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    }\n  ]\n}","type":"dataframe"}
```


- The state, area code, and phone number will not be suitable columns for providing features for machine learning, since training models only require measurable numerical data. These specific columns are used more as identifiers or columns that uniquely identify each data point.

```
cols_to_drop = ['state', 'area_code', 'phone_number']
df.drop(cols_to_drop, axis=1, inplace=True)

df.columns
```

```
Index(['account_length', 'intl_plan', 'voice_mail_plan',
       'number_vmail_messages', 'total_day_minutes',
'total_day_calls',
       'total_day_charge', 'total_eve_minutes', 'total_eve_calls',
       'total_eve_charge', 'total_night_minutes', 'total_night_calls',
       'total_night_charge', 'total_intl_minutes', 'total_intl_calls',
       'total_intl_charge', 'number_customer_service_calls',
'churned'],
      dtype='object')
```

Question 2

- Notice that some of the columns are categorical data and some are floats. These features will need to be numerically encoded using one of the methods from the lecture.

- Finally, remember from the lecture that K-nearest neighbors requires scaled data. Scale the data using one of the scaling methods discussed in the lecture.

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

df_encoded = df.copy()

# One-hot encoding; convert categorical to numerical
df_encoded = pd.get_dummies(df_encoded, columns=['intl_plan',
'voice_mail_plan'], drop_first=True)
df_encoded['churned'] = df_encoded['churned'].astype(int)

X = df_encoded.drop('churned', axis=1)
y = df_encoded['churned']

# Scale features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_scaled_df = pd.DataFrame(X_scaled, columns=X.columns)

X_scaled_df.head(5)
```


```
{"summary":"{\n  \"name\": \"X_scaled_df\",\n  \"rows\": 5000,\n
\"fields\": [\n    {\n      \"column\": \"account_length\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
1.0001000150024986,\n        \"min\": -2.5008093925546975,\n
\"max\": 3.596353704630199,\n        \"num_unique_values\": 218,\n
\"samples\": [\n          2.1350501524123313,\n
2.91609170618395,\n          0.7241363778571487\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"number_vmail_messages\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
1.0001000150025177,\n        \"min\": -0.5725491215445375,\n
\"max\": 3.266494916045202,\n        \"num_unique_values\": 48,\n
\"samples\": [\n          0.6825229676674928,\n
3.0450116061842554,\n          0.7563507376211416\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"total_day_minutes\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
1.0001000150024995,\n        \"min\": -3.3455411084050155,\n
\"max\": 3.177088402365547,\n        \"num_unique_values\": 1961,\n
\"samples\": [\n          0.5364904524860274,\n          -
1.1540118190109803,\n          -0.13896819386403889\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"total_day_calls\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
1.0001000150025017,\n        \"min\": -5.044546849416456,\n
\"max\": 3.2765090616828334,\n        \"num_unique_values\": 123,\n
```

```
\"samples\": [\n          0.855838251181222,\n          -
0.858803572924086,\n          -0.45535843784048413\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"total_day_charge\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
1.0001000150025023,\n        \"min\": -3.3456127058502783,\n
\"max\": 3.17758406422934,\n        \"num_unique_values\": 1961,\n
\"samples\": [\n          0.5359950107499769,\n          -
1.1537487308971537,\n          -0.13859260703938162\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"total_eve_minutes\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
1.000100015002502,\n        \"min\": -3.9693655963449412,\n
\"max\": 3.2260242537933137,\n        \"num_unique_values\": 1879,\n
\"samples\": [\n          -0.2836324946656531,\n
1.1170647871319508,\n          -2.60032249040717\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"total_eve_calls\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
1.0001000150025032,\n        \"min\": -5.0538946191378,\n
\"max\": 3.5213475209089706,\n        \"num_unique_values\": 126,\n
\"samples\": [\n          -2.178666372180942,\n
1.0496600805425484,\n          2.7142659077280977\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"total_eve_charge\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
1.0001000150025023,\n        \"min\": -3.969432090843908,\n
\"max\": 3.224940451669667,\n        \"num_unique_values\": 1659,\n
\"samples\": [\n          -1.6023415454746728,\n          -
0.16626006416413383,\n          -1.4068296420223465\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"total_night_minutes\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
1.000100015002499,\n        \"min\": -3.966365145052053,\n
\"max\": 3.851897077168421,\n        \"num_unique_values\": 1853,\n
\"samples\": [\n          1.642990586029674,\n
1.7340387030479383,\n          -0.7856189700879262\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"total_night_calls\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
1.0001000150024941,\n        \"min\": -5.006802257862113,\n
\"max\": 3.762187036746629,\n        \"num_unique_values\": 131,\n
\"samples\": [\n          -1.3989895195088025,\n
0.3046998291580389,\n          -1.0983384579793598\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"total_night_charge\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
1.0001000150025023,\n        \"min\": -3.9663919560690384,\n
\"max\": 3.849629307298159,\n        \"num_unique_values\": 1028,\n
```

```
\"samples\": [\n          -0.8566975085729162,\n
0.5815911538474965,\n          0.35727090374523035\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"total_intl_minutes\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
1.0001000150025012,\n        \"min\": -3.7165286661106203,\n
\"max\": 3.5269099305278204,\n        \"num_unique_values\": 170,\n
\"samples\": [\n          1.9695706322505555,\n
1.5349643164522493,\n          2.8750004568303607\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"total_intl_calls\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
1.000100015002496,\n        \"min\": -1.8054644015545165,\n
\"max\": 6.3360597757295585,\n        \"num_unique_values\": 21,\n
\"samples\": [\n          -0.5842357749619053,\n
3.8936025225443363,\n          3.4865263136801325\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"total_intl_charge\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
1.0001000150024986,\n        \"min\": -3.7175348141330447,\n
\"max\": 3.526517211172435,\n        \"num_unique_values\": 170,\n
\"samples\": [\n          1.9703875168475542,\n
1.5411103597924143,\n          2.8691865644317525\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"number_customer_service_calls\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
1.0001000150025072,\n        \"min\": -1.2022360274840573,\n
\"max\": 5.687807431097525,\n        \"num_unique_values\": 10,\n
\"samples\": [\n          3.3911262782369973,\n          -
1.2022360274840573,\n          2.6255658939501547\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"intl_plan_yes\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
1.0001000150024801,\n        \"min\": -0.3232401674851936,\n
\"max\": 3.093674922210299,\n        \"num_unique_values\": 2,\n
\"samples\": [\n          3.093674922210299,\n          -
0.3232401674851936\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"voice_mail_plan_yes\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 1.0001000150025017,\n        \"min\": -
0.5998368013025707,\n        \"max\": 1.6671201197199943,\n
\"num_unique_values\": 2,\n        \"samples\": [\n          -
0.5998368013025707,\n          1.6671201197199943\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    }\n  ]\n}","type":"dataframe","variable_name":"X_scaled_df"}
```


Question 3

- Separate the feature columns (everything except `churned`) from the label (`churned`). This will create two tables.

- Fit a K-nearest neighbors model with a value of `k=3` to this data and predict the outcome on the same data.

```
X = df_encoded.drop('churned', axis=1)
y = df_encoded['churned']

knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X, y)

predictions = knn.predict(X)
accuracy = (knn.score(X, y)) * 100

print(f"Model Accuracy: {accuracy:.2f}%")
```

```
Model Accuracy: 91.64%
```

Question 4

Ways to measure error haven't been discussed in class yet, but accuracy is an easy one to understand--it is simply the percent of labels that were correctly predicted (either true or false).

- Write a function to calculate accuracy using the actual and predicted labels.
- Using the function, calculate the accuracy of this K-nearest neighbors model on the data.

```
import numpy as np

def calculate_accuracy(y_actual, y_pred):
    actual = np.array(y_actual)
    pred = np.array(y_pred)
    return np.mean(actual == pred) * 100

accuracy_score = calculate_accuracy(y, predictions)
print(f"Model Accuracy: {accuracy_score:.2f}%")
```

```
Model Accuracy: 91.64%
```

```
def calculate_k_nearest_accuracy(actual_labels, predicted_labels):
    correct_predictions = (actual_labels == predicted_labels).sum()
    total_predictions = len(actual_labels)
    accuracy = (correct_predictions / total_predictions) * 100
    return accuracy

accuracy_score = calculate_k_nearest_accuracy(y, predictions)
print(f"Model Accuracy: {accuracy_score:.2f}%")
```

```
Model Accuracy: 91.64%
```

Question 5

- Fit the K-nearest neighbors model again with `n_neighbors=3` but this time use distance for the weights. Calculate the accuracy using the function you created above.

- Fit another K-nearest neighbors model. This time use uniform weights but set the power parameter for the Minkowski distance metric to be 1 (`p=1`) i.e. Manhattan Distance.

When weighted distances are used for part 1 of this question, a value of 1.0 should be returned for the accuracy. Why do you think this is? *Hint:* we are predicting on the data and with KNN the model *is* the data. We will learn how to avoid this pitfall in the next lecture.

```
# KNN by distance
knn_distance_weights = KNeighborsClassifier(n_neighbors=3,
weights='distance')
knn_distance_weights.fit(X_scaled, y)

predictions_distance = knn_distance_weights.predict(X_scaled)
accuracy_distance = calculate_k_nearest_accuracy(y,
predictions_distance)
print(f"Distance KNN Model Accuracy: {accuracy_distance:.2f}%")
```

```
Distance KNN Model Accuracy: 100.00%
```

```
# KNN by Manhattan distance
knn_manhattan = KNeighborsClassifier(n_neighbors=3, weights='uniform',
p=1)
knn_manhattan.fit(X_scaled, y)

predictions_manhattan = knn_manhattan.predict(X_scaled)
accuracy_manhattan = calculate_k_nearest_accuracy(y,
predictions_manhattan)
print(f"Manhattan Distance KNN Model Accuracy:
{accuracy_manhattan:.2f}%")
```

```
Manhattan Distance KNN Model Accuracy: 94.16%
```

Question 6

- Fit a K-nearest neighbors model using values of `k` (`n_neighbors`) ranging from 1 to 20. Use uniform weights (the default). The coefficient for the Minkowski distance (`p`) can be set to either 1 or 2--just be consistent. Store the accuracy and the value of `k` used from each of these fits in a list or dictionary.
- Plot (or view the table of) the `accuracy` vs `k`. What do you notice happens when `k=1`? Why do you think this is? *Hint:* it's for the same reason discussed above.

```
accuracies = []
k_values = range(1, 21)

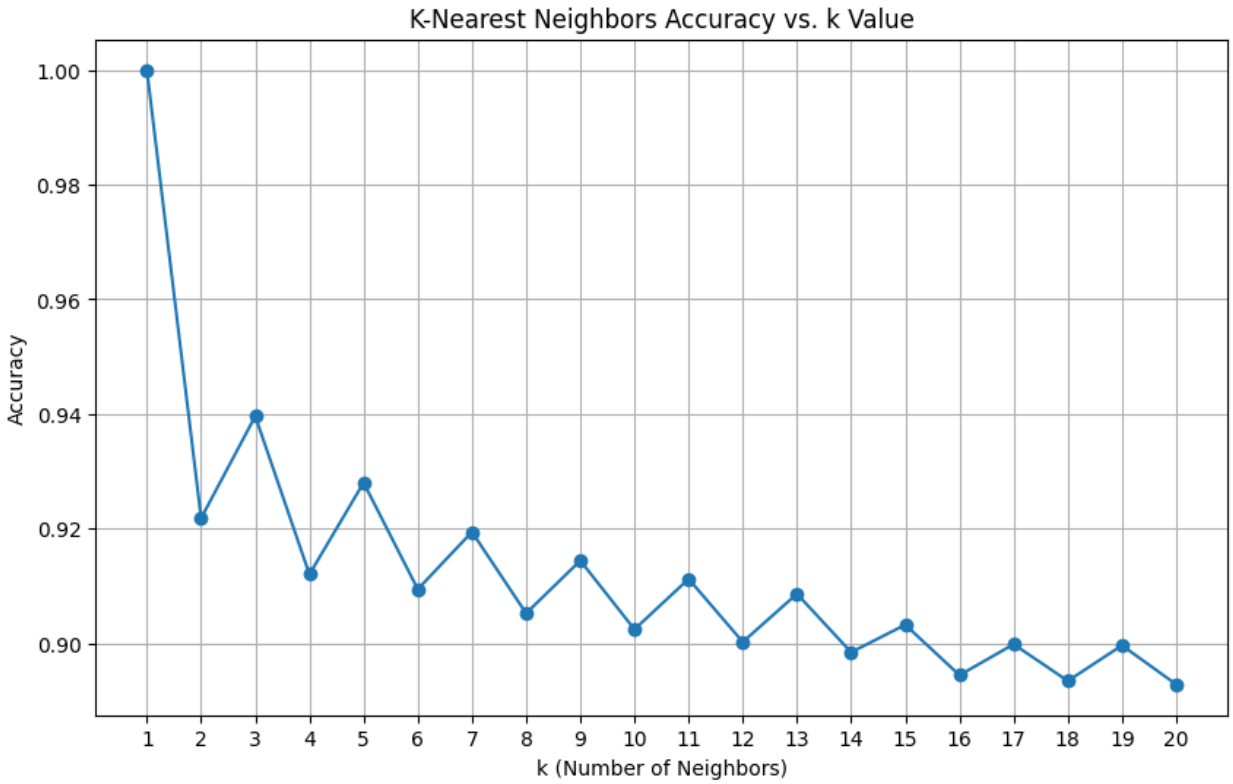
for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k, weights='uniform', p=2)
    knn.fit(X_scaled, y)
    accuracy = knn.score(X_scaled, y)
    accuracies.append(accuracy)

k_accuracy_dict = dict(zip(k_values, accuracies))
```

```python
results_df = pd.DataFrame({'k': k_values, 'accuracy': accuracies})
print("Accuracy for different k values:")
print(results_df.to_string(index=False))
```

```
Accuracy for different k values:
 k  accuracy
 1    1.0000
 2    0.9218
 3    0.9396
 4    0.9120
 5    0.9280
 6    0.9094
 7    0.9194
 8    0.9052
 9    0.9144
10    0.9024
11    0.9112
12    0.9002
13    0.9086
14    0.8984
15    0.9032
16    0.8944
17    0.8998
18    0.8934
19    0.8996
20    0.8928
```

```python
plt.figure(figsize=(10, 6))
plt.plot(k_values, accuracies, marker='o')
plt.title('K-Nearest Neighbors Accuracy vs. k Value')
plt.xlabel('k (Number of Neighbors)')
plt.ylabel('Accuracy')
plt.xticks(k_values)
plt.grid(True)
plt.show()
```



Observation: When k is equal to 1, the accuracy will be 1.0 since the closest neighbor recognized by the model is the data point itself. Therefore, the model will always correctly classify every training instance, leading to 100% accuracy.

Supplementary

1. Include Classification Report and Confusion Matrix
2. Evaluate performance of using GridSearch vs Random Search in terms of Accuracy for Training and Testing.

```
from sklearn.model_selection import GridSearchCV, RandomizedSearchCV,
train_test_split
from sklearn.metrics import classification_report, confusion_matrix,
accuracy_score
import numpy as np

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y,
test_size=0.3, random_state=42)

# Grid
param_grid = {
    'n_neighbors': np.arange(1, 21),
    'weights': ['uniform', 'distance'],
    'p': [1, 2]
}
knn = KNeighborsClassifier()
```

GridSearch

```
grid_search = GridSearchCV(knn, param_grid, cv=5, scoring='accuracy',
n_jobs=-1)
grid_search.fit(X_train, y_train)
```

```
GridSearchCV(cv=5, estimator=KNeighborsClassifier(), n_jobs=-1,
             param_grid={'n_neighbors': array([ 1,  2,  3,  4,  5,  6,
7,  8,  9, 10, 11, 12, 13, 14, 15, 16, 17,
       18, 19, 20]),
                         'p': [1, 2], 'weights': ['uniform',
'distance']},
             scoring='accuracy')
```

```
best_knn_grid = grid_search.best_estimator_
y_train_pred_grid = best_knn_grid.predict(X_train)
y_test_pred_grid = best_knn_grid.predict(X_test)
```

```
print("\nGridSearch - Training Performance:")
print(f"Accuracy: {accuracy_score(y_train, y_train_pred_grid):.4f}")
print("Classification Report:\n", classification_report(y_train,
y_train_pred_grid))
print("Confusion Matrix:\n", confusion_matrix(y_train,
y_train_pred_grid))
```

```
print("\nGridSearch - Testing Performance:")
print(f"Accuracy: {accuracy_score(y_test, y_test_pred_grid):.4f}")
print("Classification Report:\n", classification_report(y_test,
y_test_pred_grid))
print("Confusion Matrix:\n", confusion_matrix(y_test,
y_test_pred_grid))
```

```
GridSearch - Training Performance:
Accuracy: 1.0000
Classification Report:
               precision    recall  f1-score   support

           0       1.00      1.00      1.00      3000
           1       1.00      1.00      1.00       500

    accuracy                           1.00      3500
   macro avg       1.00      1.00      1.00      3500
weighted avg       1.00      1.00      1.00      3500

Confusion Matrix:
 [[3000    0]
 [   0  500]]

GridSearch - Testing Performance:
Accuracy: 0.9080
```

```
Classification Report:
              precision    recall  f1-score   support

           0       0.92      0.98      0.95      1293
           1       0.79      0.45      0.57       207

    accuracy                           0.91      1500
   macro avg       0.86      0.72      0.76      1500
weighted avg       0.90      0.91      0.90      1500

Confusion Matrix:
 [[1269   24]
 [ 114   93]]
```

RandomizedSearch

```
random_search = RandomizedSearchCV(knn, param_grid, n_iter=20, cv=5,
scoring='accuracy', random_state=42, n_jobs=-1)
random_search.fit(X_train, y_train)
```

```
RandomizedSearchCV(cv=5, estimator=KNeighborsClassifier(), n_iter=20,
n_jobs=-1,
                   param_distributions={'n_neighbors': array([ 1,  2,
3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16, 17,
       18, 19, 20]),
                                        'p': [1, 2],
                                        'weights': ['uniform',
'distance']},
                   random_state=42, scoring='accuracy')
```

```
# Evaluate RandomizedSearchCV's best estimator
best_knn_random = random_search.best_estimator_

y_train_pred_random = best_knn_random.predict(X_train)
y_test_pred_random = best_knn_random.predict(X_test)

print("\nRandomSearch - Training Performance:")
print(f"Accuracy: {accuracy_score(y_train, y_train_pred_random):.4f}")
print("Classification Report:\n", classification_report(y_train,
y_train_pred_random))
print("Confusion Matrix:\n", confusion_matrix(y_train,
y_train_pred_random))

print("\nRandomSearch - Testing Performance:")
print(f"Accuracy: {accuracy_score(y_test, y_test_pred_random):.4f}")
print("Classification Report:\n", classification_report(y_test,
y_test_pred_random))
print("Confusion Matrix:\n", confusion_matrix(y_test,
y_test_pred_random))
```

```
RandomSearch - Training Performance:
Accuracy: 1.0000
Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00      3000
           1       1.00      1.00      1.00       500

    accuracy                           1.00      3500
   macro avg       1.00      1.00      1.00      3500
weighted avg       1.00      1.00      1.00      3500

Confusion Matrix:
 [[3000    0]
 [   0  500]]

RandomSearch - Testing Performance:
Accuracy: 0.9000
Classification Report:
              precision    recall  f1-score   support

           0       0.91      0.99      0.94      1293
           1       0.82      0.35      0.49       207

    accuracy                           0.90      1500
   macro avg       0.86      0.67      0.72      1500
weighted avg       0.89      0.90      0.88      1500

Confusion Matrix:
 [[1277   16]
 [ 134   73]]

print("\n--- Comparison ---")
print(f"GridSearch Test Accuracy: {accuracy_score(y_test,
y_test_pred_grid):.4f}")
print(f"RandomSearch Test Accuracy: {accuracy_score(y_test,
y_test_pred_random):.4f}")


--- Comparison ---
GridSearch Test Accuracy: 0.9080
RandomSearch Test Accuracy: 0.9000
```